

Step 1: Importing libraries

```
In [59]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

Step 2: Data Loading

```
In [118... # — Update paths to your actual CSV file locations —
deliveries_raw = pd.read_csv('deliveries.csv')
matches_raw    = pd.read_csv('matches.csv')
players_raw    = pd.read_csv('players.csv')

print(f'deliveries : {deliveries_raw.shape}')
print(f'matches    : {matches_raw.shape}')
print(f'players     : {players_raw.shape}')
```

```
deliveries : (13500, 7)
matches    : (220, 7)
players    : (2300, 4)
```

Step 2.1: Data Understanding

```
In [119... # — Quick Look at each table —
for name, df in [('deliveries', deliveries_raw),
                 ('matches', matches_raw),
                 ('players', players_raw)]:
    print(f'\n{'='*45}')
    print(f'  TABLE: {name.upper()}')
    print(f'{'='*45}')
    print(df.info())
    print(f'\nFirst 3 rows:')
    display(df.head(3))
```

=====

TABLE: DELIVERIES

=====

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 13500 entries, 0 to 13499

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	match_id	13500 non-null	int64
1	inning	13500 non-null	int64
2	over	13500 non-null	int64
3	ball	13500 non-null	int64
4	batsman_runs	11637 non-null	float64
5	bowler_runs	11523 non-null	float64
6	is_wicket	9053 non-null	float64

dtypes: float64(3), int64(4)

memory usage: 738.4 KB

None

First 3 rows:

	match_id	inning	over	ball	batsman_runs	bowler_runs	is_wicket
0	969	1	6	3	1.0	2.0	0.0
1	84	2	11	4	4.0	6.0	1.0
2	223	1	7	5	0.0	0.0	1.0

=====

TABLE: MATCHES

=====

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 220 entries, 0 to 219

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	match_id	220 non-null	int64
1	season	220 non-null	int64
2	team1	220 non-null	object
3	team2	220 non-null	object
4	venue	220 non-null	object
5	winner	186 non-null	object
6	match_date	220 non-null	object

dtypes: int64(2), object(5)

memory usage: 12.2+ KB

None

First 3 rows:

	match_id	season	team1	team2	venue	winner	match_date
0	1	2024	LSG	DC	Kolkata	LSG	2020-03-21
1	2	2024	LSG	PBKS	Mumbai	LSG	2020-03-22
2	3	2022	GT	PBKS	Hyderabad	PBKS	2020-03-23

```
=====
TABLE: PLAYERS
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2300 entries, 0 to 2299
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   player_id    2300 non-null   int64
1   player_name  2300 non-null   object
2   team         2300 non-null   object
3   role         1830 non-null   object
dtypes: int64(1), object(3)
memory usage: 72.0+ KB
None
```

First 3 rows:

	player_id	player_name	team	role
0	1	Player_1	RCB	Batsman
1	2	Player_2	PBKS	Wicket-Keeper
2	3	Player_3	SRH	Batsman

Step 3: Data Cleaning

Step 3.1: Null Value Summary (Before)

```
In [120... def null_summary(df, name):
    """Show null count + percentage per column."""
    nulls = df.isnull().sum()
    pct = (nulls / len(df) * 100).round(2)
    summary = pd.DataFrame({'null_count': nulls, 'null_%': pct})
    summary = summary[summary['null_count'] > 0]
    print(f'\n— {name} : columns with nulls —')
    print(summary if not summary.empty else ' No nulls found ✅')

null_summary(deliveries_raw, 'deliveries')
null_summary(matches_raw, 'matches')
null_summary(players_raw, 'players')
```

```
— deliveries : columns with nulls —
      null_count  null_%
batsman_runs      1863   13.80
bowler_runs       1977   14.64
is_wicket         4447   32.94

— matches : columns with nulls —
      null_count  null_%
winner           34   15.45

— players : columns with nulls —
      null_count  null_%
role            470   20.43
```

Step 3.2: Clean: DELIVERIES

```
In [124... deliveries = deliveries_raw.copy()

# — 1. Remove exact duplicates —
before = len(deliveries)
deliveries.drop_duplicates(inplace=True)
print(f'Duplicates removed : {before - len(deliveries)}')

# — 2. Handle nulls —
# Runs columns: missing likely means 0 (no runs scored on that delivery)
deliveries['batsman_runs'] = deliveries['batsman_runs'].fillna(0)
deliveries['bowler_runs'] = deliveries['bowler_runs'].fillna(0)

# is_wicket: missing likely means no wicket (0)
deliveries['is_wicket'] = deliveries['is_wicket'].fillna(0)

# — 3. Fix dtypes (runs & wicket should be int, not float) —
deliveries['batsman_runs'] = deliveries['batsman_runs'].astype(int)
deliveries['bowler_runs'] = deliveries['bowler_runs'].astype(int)
deliveries['is_wicket'] = deliveries['is_wicket'].astype(int)

print(f'\nDeliveries after cleaning : {deliveries.shape}')
print('Remaining nulls :', deliveries.isnull().sum().sum())
```

Duplicates removed : 1501

Deliveries after cleaning : (11999, 7)

Remaining nulls : 0

Step 3.3: Clean: MATCHES

```
In [126... matches = matches_raw.copy()

# — 1. Remove exact duplicates —
before = len(matches)
matches.drop_duplicates(inplace=True)
print(f'Duplicates removed : {before - len(matches)}')

# — 2. Handle nulls —
# winner: fill with 'No Result' for tied/abandoned matches
matches.fillna({
    'winner': 'No Result'
}, inplace=True)

# — 3. Fix dtypes —
# match_date: convert from object string to datetime
matches['match_date'] = pd.to_datetime(matches['match_date'], errors='coerce')

# Standardise team name strings (strip accidental whitespace)
for col in ['team1', 'team2', 'winner', 'venue']:
    matches[col] = matches[col].str.strip()

print(f'\nMatches after cleaning : {matches.shape}')
print('Remaining nulls :', matches.isnull().sum().sum())
```

Duplicates removed : 14

Matches after cleaning : (206, 7)

Remaining nulls : 0

Step 3.4: Clean: Players

```
In [128... # — 1. Remove exact duplicates —
before = len(players)
players.drop_duplicates(inplace=True)
print(f'Duplicates removed : {before - len(players)}')

# — 2. Handle nulls —
# role: fill with 'Unknown' - role info simply not available
players['role'] = players['role'].fillna('Unknown')

# — 3. Standardise strings —
players['player_name'] = players['player_name'].str.strip()
players['team'] = players['team'].str.strip()
players['role'] = players['role'].str.strip()

print(f'\nPlayers after cleaning : {players.shape}')
print('Remaining nulls :', players.isnull().sum().sum())
```

Duplicates removed : 0

Players after cleaning : (2000, 4)

Remaining nulls : 0

Step 3.5: Before vs After Summary

```
In [129... summary = pd.DataFrame({
    'Table' : ['deliveries', 'matches', 'players'],
    'Rows (before)': [len(deliveries_raw), len(matches_raw), len(players_raw)],
    'Rows (after)' : [len(deliveries), len(matches), len(players)],
    'Nulls (before)': [
        deliveries_raw.isnull().sum().sum(),
        matches_raw.isnull().sum().sum(),
        players_raw.isnull().sum().sum()
    ],
    'Nulls (after)' : [
        deliveries.isnull().sum().sum(),
        matches.isnull().sum().sum(),
        players.isnull().sum().sum()
    ]
})
display(summary)
```

	Table	Rows (before)	Rows (after)	Nulls (before)	Nulls (after)
0	deliveries	13500	11999	8287	0
1	matches	220	206	34	0
2	players	2300	2000	470	0

Step 4: Feature Engineering(Business rules)

Step 4.1: PLAYERS — Business Rules

In [133...

```
# — Rule 1: player_name must not be blank —
blank_name = players[players['player_name'].str.strip() == '']
print(f'Rule 1 | blank player_name : {len(blank_name)} rows')

# — Rule 2: team must not be blank —
blank_team = players[players['team'].str.strip() == '']
print(f'Rule 2 | blank team : {len(blank_team)} rows')

# — Rule 3: role must be a known cricket role —
valid_roles = ['Batsman', 'Bowler', 'All-Rounder', 'Wicket-Keeper', 'Unknown']
invalid_role = players[~players['role'].isin(valid_roles)]
print(f'Rule 3 | unexpected role : {len(invalid_role)} rows')
if len(invalid_role) > 0:
    print(' Unique bad values:', invalid_role['role'].unique())

# — Rule 4: player_id must be unique —
dupe_ids = players[players.duplicated(subset=['player_id'], keep=False)]
print(f'Rule 4 | duplicate IDs : {len(dupe_ids)} rows')
```

```
Rule 1 | blank player_name : 0 rows
Rule 2 | blank team : 0 rows
Rule 3 | unexpected role : 0 rows
Rule 4 | duplicate IDs : 0 rows
```

Step 4.2: MATCHES — Business Rules

In [131...

```
# — Rule 1: player_name must not be blank —
blank_name = players[players['player_name'].str.strip() == '']
print(f'Rule 1 | blank player_name : {len(blank_name)} rows')

# — Rule 2: team must not be blank —
blank_team = players[players['team'].str.strip() == '']
print(f'Rule 2 | blank team : {len(blank_team)} rows')

# — Rule 3: role must be a known cricket role —
valid_roles = ['Batsman', 'Bowler', 'All-Rounder', 'Wicket-Keeper', 'Unknown']
invalid_role = players[~players['role'].isin(valid_roles)]
print(f'Rule 3 | unexpected role : {len(invalid_role)} rows')
if len(invalid_role) > 0:
    print(' Unique bad values:', invalid_role['role'].unique())

# — Rule 4: player_id must be unique —
dupe_ids = players[players.duplicated(subset=['player_id'], keep=False)]
print(f'Rule 4 | duplicate IDs : {len(dupe_ids)} rows')
```

```
Rule 1 | blank player_name : 0 rows
Rule 2 | blank team : 0 rows
Rule 3 | unexpected role : 0 rows
Rule 4 | duplicate IDs : 0 rows
```

Step 4.3: DELIVERIES — Business Rules

```
In [130]: # — Rule 1: batsman_runs must be 0 or more —
invalid_bat = deliveries[deliveries['batsman_runs'] < 0]
print(f'Rule 1 | batsman_runs < 0 : {len(invalid_bat)} rows')

# — Rule 2: bowler_runs must be 0 or more —
invalid_bowl = deliveries[deliveries['bowler_runs'] < 0]
print(f'Rule 2 | bowler_runs < 0 : {len(invalid_bowl)} rows')

# — Rule 3: is_wicket must be only 0 or 1 —
invalid_wkt = deliveries[~deliveries['is_wicket'].isin([0, 1])]
print(f'Rule 3 | is_wicket not 0/1: {len(invalid_wkt)} rows')

# — Rule 4: over must be between 1 and 20 (T20 format) —
invalid_over = deliveries[~deliveries['over'].between(1, 20)]
print(f'Rule 4 | over not 1-20 : {len(invalid_over)} rows')

# — Rule 5: ball must be between 1 and 6 —
invalid_ball = deliveries[~deliveries['ball'].between(1, 6)]
print(f'Rule 5 | ball not 1-6 : {len(invalid_ball)} rows')
```

```
Rule 1 | batsman_runs < 0 : 0 rows
Rule 2 | bowler_runs < 0 : 0 rows
Rule 3 | is_wicket not 0/1: 0 rows
Rule 4 | over not 1-20 : 0 rows
Rule 5 | ball not 1-6 : 0 rows
```

Step 4.4: Total Wickets Per Match

```
In [78]: match_wickets = deliveries.groupby('match_id')['is_wicket'].sum()
match_wickets
```

```
Out[78]: match_id
1         3.0
2         1.0
3         3.0
4         2.0
5         3.0
...
1996      2.0
1997      1.0
1998      5.0
1999      1.0
2000      2.0
Name: is_wicket, Length: 1996, dtype: float64
```

Step 5: Data loading to sql

Step 5.1: Connecting to MySQL

```
In [84]: from sqlalchemy import create_engine

engine = create_engine(
```

```
"mysql+pymysql://root:root123@localhost/ipl_analysis"
)
```

Step 5.2: Loading data into sql tables

```
In [85]: deliveries.to_sql(
        'deliveries',
        con=engine,
        if_exists='replace',
        index=False
    )

print("Data Loaded Successfully")
```

Data Loaded Successfully

```
In [86]: matches.to_sql(
        'matches',
        con=engine,
        if_exists='replace',
        index=False
    )

print("Data Loaded Successfully")
```

Data Loaded Successfully

```
In [87]: players.to_sql(
        'players',
        con=engine,
        if_exists='replace',
        index=False
    )

print("Data Loaded Successfully")
```

Data Loaded Successfully

Step 5.3: Analytics using queries

```
In [135... def run_query(query):
            return pd.read_sql(query, engine)
```

Step 5.3.1: Team Analytics

```
In [136... # — Q1: Team with highest wins across all seasons —————
q1 = run_query("""
    SELECT
        winner                AS team,
        COUNT(*)              AS total_wins
    FROM    matches
    WHERE   winner IS NOT NULL
    AND     winner != 'No Result'
    GROUP   BY winner
    ORDER   BY total_wins DESC;
""")

print('Q1 — All-time win leaderboard')
```



```
display(q1)
print(f"\n Most wins: {q1.iloc[0]['team']} ({q1.iloc[0]['total_wins']} wins)")
```

Q1 – All-time win leaderboard

	team	total_wins
0	PBKS	23
1	GT	20
2	SRH	20
3	CSK	19
4	RR	18
5	DC	17
6	MI	16
7	KKR	16
8	LSG	13
9	RCB	11

Most wins: PBKS (23 wins)

In [137...

```
# — Q2: Team wins per season
q2 = run_query("""
    SELECT
        season,
        winner                AS team,
        COUNT(*)              AS wins
    FROM    matches
    WHERE   winner IS NOT NULL
    AND     winner != 'No Result'
    GROUP   BY season, winner
    ORDER   BY season ASC, wins DESC;
""")

print('Q2 – Wins per team per season')
display(q2)
```

Q2 – Wins per team per season

	season	team	wins
0	2020	PBKS	6
1	2020	GT	6
2	2020	RR	6
3	2020	SRH	5
4	2020	DC	5
5	2020	KKR	5
6	2020	RCB	4
7	2020	CSK	3
8	2020	MI	2
9	2021	DC	4
10	2021	LSG	4
11	2021	RR	3
12	2021	SRH	3
13	2021	PBKS	3
14	2021	CSK	2
15	2021	GT	2
16	2021	KKR	2
17	2021	MI	1
18	2021	RCB	1
19	2022	SRH	7
20	2022	MI	5
21	2022	DC	4
22	2022	KKR	4
23	2022	PBKS	3
24	2022	CSK	3
25	2022	RCB	3
26	2022	GT	3
27	2022	LSG	1
28	2023	CSK	7
29	2023	SRH	5
30	2023	LSG	4
31	2023	MI	4
32	2023	PBKS	4

	season	team	wins
33	2023	GT	4
34	2023	DC	3
35	2023	RR	3
36	2023	RCB	2
37	2023	KKR	2
38	2024	PBKS	7
39	2024	RR	6
40	2024	GT	5
41	2024	CSK	4
42	2024	LSG	4
43	2024	MI	4
44	2024	KKR	3
45	2024	RCB	1
46	2024	DC	1

In [138...

```
# — Q3: Which team won the most matches in each season (season champion) —
q3 = run_query("""
    SELECT season, team, wins
    FROM (
        SELECT
            season,
            winner          AS team,
            COUNT(*)        AS wins,
            RANK() OVER (PARTITION BY season ORDER BY COUNT(*) DESC) AS rnk
        FROM matches
        WHERE winner IS NOT NULL
        AND   winner != 'No Result'
        GROUP BY season, winner
    ) ranked
    WHERE rnk = 1
    ORDER BY season;
""")

print('Q3 – Best team per season')
display(q3)
```

Q3 – Best team per season

	season	team	wins
0	2020	PBKS	6
1	2020	GT	6
2	2020	RR	6
3	2021	DC	4
4	2021	LSG	4
5	2022	SRH	7
6	2023	CSK	7
7	2024	PBKS	7

Step 5.3.2: Match Insights

In [139...

```
# — Q4: Average runs per match —————
q4 = run_query("""
    SELECT
        ROUND(AVG(match_runs), 2) AS avg_runs_per_match,
        MAX(match_runs)          AS highest_match_runs,
        MIN(match_runs)          AS lowest_match_runs
    FROM (
        SELECT
            match_id,
            SUM(batsman_runs) AS match_runs
        FROM deliveries
        WHERE inning IN (1, 2)
        GROUP BY match_id
    ) match_totals;
""")

print('Q4 – Average runs per match')
display(q4)
```

Q4 – Average runs per match

	avg_runs_per_match	highest_match_runs	lowest_match_runs
0	13.89	45.0	0.0

In [140...

```
# — Q5: Average runs per match – season wise —————
q5 = run_query("""
    SELECT
        m.season,
        COUNT(DISTINCT d.match_id) AS total
        SUM(d.batsman_runs)        AS total
        ROUND(SUM(d.batsman_runs) / COUNT(DISTINCT d.match_id), 2) AS avg_ru
    FROM deliveries d
    JOIN matches m ON d.match_id = m.match_id
    WHERE d.inning IN (1, 2)
    GROUP BY m.season
    ORDER BY m.season;
""")
```

```
print('Q5 – Avg runs per match by season')
display(q5)
```

Q5 – Avg runs per match by season

	season	total_matches	total_runs	avg_runs_per_match
0	2020	50	769.0	15.38
1	2021	36	617.0	17.14
2	2022	37	542.0	14.65
3	2023	39	591.0	15.15
4	2024	38	536.0	14.11

In [141...

```
# — Q6: Matches with No Result
q6 = run_query("""
    SELECT
        match_id,
        season,
        team1,
        team2,
        venue,
        match_date
    FROM matches
    WHERE winner IS NULL
    OR winner = 'No Result'
    ORDER BY season, match_date;
""")

print(f'Q6 – Matches with No Result: {len(q6)} matches')
display(q6)
```

Q6 – Matches with No Result: 33 matches

	match_id	season	team1	team2	venue	match_date
0	46	2020	MI	RR	Hyderabad	2020-05-05
1	79	2020	RCB	PBKS	Mumbai	2020-06-07
2	126	2020	CSK	SRH	Kolkata	2020-07-24
3	136	2020	MI	RR	Bangalore	2020-08-03
4	138	2020	MI	MI	Kolkata	2020-08-05
5	151	2020	SRH	PBKS	Hyderabad	2020-08-18
6	163	2020	PBKS	CSK	Mumbai	2020-08-30
7	199	2020	RR	PBKS	Mumbai	2020-10-05
8	27	2021	PBKS	SRH	Delhi	2020-04-16
9	45	2021	GT	KKR	Chennai	2020-05-04
10	47	2021	CSK	MI	Bangalore	2020-05-06
11	69	2021	CSK	CSK	Hyderabad	2020-05-28
12	75	2021	PBKS	SRH	Mumbai	2020-06-03
13	108	2021	RR	LSG	Ahmedabad	2020-07-06
14	172	2021	RR	CSK	Hyderabad	2020-09-08
15	177	2021	SRH	PBKS	Ahmedabad	2020-09-13
16	179	2021	GT	LSG	Kolkata	2020-09-15
17	191	2021	RCB	MI	Chennai	2020-09-27
18	192	2021	RR	LSG	Hyderabad	2020-09-28
19	30	2022	LSG	DC	Kolkata	2020-04-19
20	54	2022	DC	GT	Ahmedabad	2020-05-13
21	60	2022	RR	GT	Ahmedabad	2020-05-19
22	88	2022	PBKS	LSG	Hyderabad	2020-06-16
23	127	2022	RR	SRH	Hyderabad	2020-07-25
24	174	2022	CSK	GT	Hyderabad	2020-09-10
25	78	2023	RR	GT	Bangalore	2020-06-06
26	158	2023	RCB	CSK	Chennai	2020-08-25
27	176	2023	RCB	MI	Hyderabad	2020-09-12
28	17	2024	RR	SRH	Delhi	2020-04-06
29	22	2024	RR	LSG	Chennai	2020-04-11
30	111	2024	PBKS	CSK	Hyderabad	2020-07-09
31	135	2024	KKR	GT	Delhi	2020-08-02
32	182	2024	RR	CSK	Ahmedabad	2020-09-18

In [142...

```
# — Q7: No Result count per season —————
q7 = run_query("""
SELECT
    season,
    COUNT(*) AS no_result_count
FROM matches
WHERE winner IS NULL
OR winner = 'No Result'
GROUP BY season
ORDER BY season;
""")

print('Q7 – No Result matches per season')
display(q7)
```

Q7 – No Result matches per season

	season	no_result_count
0	2020	8
1	2021	11
2	2022	6
3	2023	3
4	2024	5

Step 5.3.3: Player & Ball Analytics

In [143...

```
# — Q8: Runs distribution per match —————
q8 = run_query("""
SELECT
    d.match_id,
    m.season,
    m.team1,
    m.team2,
    SUM(d.batsman_runs) AS total_batsman_runs,
    SUM(d.bowler_runs) AS total_bowler_runs,
    SUM(d.batsman_runs
        + d.bowler_runs) AS total_runs
FROM deliveries d
JOIN matches m ON d.match_id = m.match_id
WHERE d.inning IN (1, 2)
GROUP BY d.match_id, m.season, m.team1, m.team2
ORDER BY total_batsman_runs DESC;
""")

print(f'Q8 – Runs distribution across {len(q8)} matches')
print(f'    Mean  : {q8["total_batsman_runs"].mean():.1f}')
print(f'    Median: {q8["total_batsman_runs"].median():.1f}')
print(f'    Max   : {q8["total_batsman_runs"].max():.1f}')
print(f'    Min   : {q8["total_batsman_runs"].min():.1f}')
display(q8.head(10))
```

Q8 – Runs distribution across 200 matches

Mean : 15.3

Median: 13.0

Max : 58.0

Min : 0.0

	match_id	season	team1	team2	total_batsman_runs	total_bowler_runs	total_runs
0	35	2023	MI	CSK	58.0	16.0	74.0
1	98	2020	CSK	RR	43.0	30.0	73.0
2	52	2022	SRH	PBKS	40.0	30.0	70.0
3	199	2020	RR	PBKS	39.0	28.0	67.0
4	83	2024	RR	MI	38.0	22.0	60.0
5	171	2021	LSG	GT	38.0	20.0	58.0
6	51	2022	GT	SRH	36.0	42.0	78.0
7	72	2024	GT	RCB	34.0	17.0	51.0
8	63	2022	RCB	MI	33.0	19.0	52.0
9	84	2020	RCB	KKR	32.0	36.0	68.0

In [144...

```
# — Q9: Wickets per match —————
q9 = run_query("""
SELECT
    d.match_id,
    m.season,
    m.team1,
    m.team2,
    SUM(d.is_wicket)          AS total_wickets
FROM   deliveries d
JOIN   matches    m ON d.match_id = m.match_id
GROUP BY d.match_id, m.season, m.team1, m.team2
ORDER BY total_wickets DESC;
""")

print(f'Q9 – Wickets per match')
print(f'    Avg wickets/match : {q9["total_wickets"].mean():.1f}')
print(f'    Max wickets       : {q9["total_wickets"].max():.1f}')
display(q9.head(10))
```

Q9 – Wickets per match

Avg wickets/match : 2.0

Max wickets : 8.0

	match_id	season	team1	team2	total_wickets
0	35	2023	MI	CSK	8.0
1	84	2020	RCB	KKR	7.0
2	159	2022	CSK	GT	6.0
3	83	2024	RR	MI	6.0
4	51	2022	GT	SRH	6.0
5	194	2024	SRH	PBKS	6.0
6	179	2021	GT	LSG	6.0
7	93	2024	PBKS	SRH	6.0
8	143	2020	CSK	SRH	5.0
9	69	2021	CSK	CSK	5.0

In [145...

```
# — Q10: Average wickets per match by season —————
q10 = run_query("""
SELECT
    m.season,
    COUNT(DISTINCT d.match_id)           AS match
    SUM(d.is_wicket)                     AS total
    ROUND(SUM(d.is_wicket) / COUNT(DISTINCT d.match_id), 2) AS avg_wi
FROM   deliveries d
JOIN   matches    m ON d.match_id = m.match_id
GROUP BY m.season
ORDER BY m.season;
""")

print('Q10 – Wickets per match by season')
display(q10)
```

Q10 – Wickets per match by season

	season	matches	total_wickets	avg_wickets_per_match
0	2020	50	88.0	1.76
1	2021	36	68.0	1.89
2	2022	37	81.0	2.19
3	2023	39	87.0	2.23
4	2024	38	82.0	2.16

In [146...

```
# — Q11: Top 10 high-scoring matches —————
q11 = run_query("""
SELECT
    d.match_id,
    m.season,
    m.team1,
    m.team2,
    m.venue,
    m.winner,
    SUM(d.batsman_runs) AS total_batsman_runs

```

```

FROM deliveries d
JOIN matches m ON d.match_id = m.match_id
WHERE d.inning IN (1, 2)
GROUP BY d.match_id, m.season, m.team1, m.team2, m.venue, m.winner
ORDER BY total_batsman_runs DESC
LIMIT 10;
"""

print('Q11 – Top 10 high-scoring matches')
display(q11)

```

Q11 – Top 10 high-scoring matches

	match_id	season	team1	team2	venue	winner	total_batsman_runs
0	98	2020	CSK	RR	Mumbai	RR	43.0
1	52	2022	SRH	PBKS	Ahmedabad	SRH	40.0
2	199	2020	RR	PBKS	Mumbai	No Result	39.0
3	171	2021	LSG	GT	Mumbai	LSG	38.0
4	72	2024	GT	RCB	Delhi	GT	34.0
5	63	2022	RCB	MI	Bangalore	RCB	33.0
6	84	2020	RCB	KKR	Hyderabad	RCB	32.0
7	4	2020	PBKS	CSK	Kolkata	PBKS	32.0
8	73	2023	KKR	SRH	Bangalore	SRH	32.0
9	99	2021	DC	MI	Bangalore	DC	31.0

In [147...

```

# — Q12: Top 10 run scorers (via player_id joined with players table) —
q12 = run_query("""
SELECT
    p.player_name,
    p.team,
    p.role,
    SUM(d.batsman_runs) AS total_runs,
    COUNT(d.batsman_runs) AS balls_faced,
    SUM(CASE WHEN d.batsman_runs = 4 THEN 1 ELSE 0 END) AS fours,
    SUM(CASE WHEN d.batsman_runs = 6 THEN 1 ELSE 0 END) AS sixes,
    ROUND(
        SUM(d.batsman_runs) * 100.0 / COUNT(d.batsman_runs)
    , 2) AS strike_rate
FROM deliveries d
JOIN players p ON d.match_id = p.player_id
WHERE d.inning IN (1, 2)
GROUP BY p.player_name, p.team, p.role
ORDER BY total_runs DESC
LIMIT 10;
""")

print('Q12 – Top 10 run scorers with player details')
display(q12)

```

Q12 – Top 10 run scorers with player details

	player_name	team	role	total_runs	balls_faced	fours	sixes	strike_rate
0	Player_316	DC	Wicket-Keeper	45.0	15	3.0	3.0	300.00
1	Player_1428	DC	Unknown	45.0	11	2.0	4.0	409.09
2	Player_726	CSK	Unknown	43.0	14	4.0	3.0	307.14
3	Player_98	SRH	Batsman	43.0	13	2.0	4.0	330.77
4	Player_557	DC	All-Rounder	42.0	12	1.0	4.0	350.00
5	Player_896	SRH	Wicket-Keeper	41.0	12	2.0	3.0	341.67
6	Player_879	CSK	Batsman	41.0	10	3.0	4.0	410.00
7	Player_52	RR	All-Rounder	40.0	13	3.0	2.0	307.69
8	Player_259	RCB	Unknown	40.0	11	5.0	3.0	363.64
9	Player_199	KKR	Batsman	39.0	12	1.0	3.0	325.00

In [155...

```

q13 = run_query("""
SELECT
    CASE
        WHEN `over` <= 6 THEN 'Powerplay (1-6)'
        WHEN `over` <= 15 THEN 'Middle (7-15)'
        ELSE 'Death (16-20)'
    END AS over_phase,

    SUM(batsman_runs) AS total_runs,

    ROUND(
        AVG(batsman_runs),3
    ) AS avg_runs_per_ball,

    SUM(is_wicket) AS total_wickets,

    ROUND(
        SUM(is_wicket) * 100.0 / COUNT(*),
        3
    ) AS wicket_rate_pct

FROM deliveries

WHERE inning IN (1,2)

GROUP BY over_phase

ORDER BY FIELD(
    over_phase,
    'Powerplay (1-6)',
    'Middle (7-15)',
    'Death (16-20)'
);
""")

display(q13)

```

	over_phase	total_runs	avg_runs_per_ball	total_wickets	wicket_rate_pct
0	Powerplay (1-6)	8541.0	2.349	1196.0	32.893
1	Middle (7-15)	12301.0	2.284	1863.0	34.590
2	Death (16-20)	6877.0	2.312	991.0	33.322

In [151...

```
# — Q14: Innings 1 vs Innings 2 – runs and wickets comparison
q14 = run_query("""
    SELECT
        inning,
        SUM(batsman_runs) AS total_runs,
        ROUND(AVG(batsman_runs), 3) AS avg_runs_per_ball,
        SUM(is_wicket) AS total_wickets,
        COUNT(*) AS total_balls
    FROM deliveries
    WHERE inning IN (1, 2)
    GROUP BY inning
    ORDER BY inning;
""")

print('Q14 – Innings 1 vs 2 comparison')
display(q14)
```

Q14 – Innings 1 vs 2 comparison

	inning	total_runs	avg_runs_per_ball	total_wickets	total_balls
0	1	13667.0	2.309	2042.0	5920
1	2	14052.0	2.313	2008.0	6076

In [152...

```
# — Q15: Top 10 highest scoring venues
q15 = run_query("""
    SELECT
        m.venue,
        COUNT(DISTINCT d.match_id) AS match_count,
        SUM(d.batsman_runs) AS total_runs,
        ROUND(SUM(d.batsman_runs) / COUNT(DISTINCT d.match_id), 1) AS avg_runs_per_match
    FROM deliveries d
    JOIN matches m ON d.match_id = m.match_id
    WHERE d.inning IN (1, 2)
    GROUP BY m.venue
    ORDER BY avg_runs_per_match DESC
    LIMIT 10;
""")

print('Q15 – Top 10 highest scoring venues')
display(q15)
```

Q15 – Top 10 highest scoring venues

	venue	matches_played	total_runs	avg_runs_per_match
0	Bangalore	33	587.0	17.8
1	Mumbai	30	521.0	17.4
2	Chennai	25	397.0	15.9
3	Kolkata	24	362.0	15.1
4	Delhi	30	443.0	14.8
5	Hyderabad	32	430.0	13.4
6	Ahmedabad	26	315.0	12.1

In [154...

```

print('=' * 55)
print('  IPL SQL ANALYTICS – QUICK SUMMARY')
print('=' * 55)
print(f"  Most wins (all-time) : {q1.iloc[0]['team']} – {q1.iloc[0]['total_wins']}")
print(f"  Avg runs per match   : {q4.iloc[0]['avg_runs_per_match']}")
print(f"  No-Result matches    : {len(q6)}")
print(f"  Avg wickets/match    : {q9['total_wickets'].mean():.1f}")
print(f"  Highest match score  : {q11.iloc[0]['total_batsman_runs']} runs")
print(f"      ({q11.iloc[0]['team1']} vs {q11.iloc[0]['team2']}, {q11.iloc[0]['season']})")
print('=' * 55)

```

```

=====
IPL SQL ANALYTICS – QUICK SUMMARY
=====
Most wins (all-time) : PBKS – 23 wins
Avg runs per match   : 13.89
No-Result matches    : 33
Avg wickets/match    : 2.0
Highest match score  : 43.0 runs
      (CSK vs RR, 2020)
=====

```

Step 6: Exploratory Data Analysis

Step 6.1: Team wins comparison

In [110...

```

wins = matches[matches['winner'].notna()].groupby('winner').size().sort_values(ascending=False)
print(f"Top 5:\n{wins.head()}\nBest: {wins.idxmax()} ({wins.max()}) | Least: {wins.idxmin()} ({wins.min()})")

fig, axes = plt.subplots(1, 2, figsize=(16, 5), facecolor='#F6F8FA')
fig.suptitle(' Team with Highest Wins Across Seasons', fontsize=14, fontweight='bold')

# Plot 1
top10 = wins.head(10)
colors = ['#3B82F6', '#10B981', '#F59E0B', '#EF4444', '#8B5CF6', '#F97316', '#06B6D4', '#FFD700', '#FF69B4', '#FF6347']
axes[0].barh(top10.index, top10.values, color=colors[:len(top10)], height=0.6, edgecolor='black')
for i, (team, wins) in enumerate(top10.items()):
    axes[0].text(v+0.3, i, str(wins), va='center', fontsize=9, fontweight='bold')
axes[0].invert_yaxis()
axes[0].set(title='Total Wins by Team', xlabel='Wins')
axes[0].tick_params(axis='y', labelsize=8)
axes[0].grid(axis='x', alpha=0.3); axes[0].set_facecolor('#FFFFFF')

```

```
# Plot 2
top5 = wins.head(5).index.tolist()
wins_season = matches[matches['winner'].isin(top5)].groupby(['season', 'winner'])
wins_season = wins_season[[t for t in top5 if t in wins_season.columns]]
x, bottom = np.arange(len(wins_season)), np.zeros(len(wins_season))
for i, team in enumerate(wins_season.columns):
    axes[1].bar(x, wins_season[team].values, bottom=bottom, width=0.6,
               color=['#F5C518', '#3B82F6', '#7C3AED', '#0EA5E9', '#EF4444'][i],
               edgecolor='white', label=team.split()[0])
    bottom += wins_season[team].values
axes[1].set_xticks(x); axes[1].set_xticklabels(wins_season.index)
axes[1].set(title='Top 5 Teams — Wins per Season', xlabel='Season', ylabel='Wins')
axes[1].legend(ncol=3, fontsize=7, loc='upper left')
axes[1].grid(axis='y', alpha=0.3); axes[1].set_facecolor('#FFFFFF')

plt.tight_layout(); plt.show()
```

Top 5:

winner

No Result 33

PBKS 23

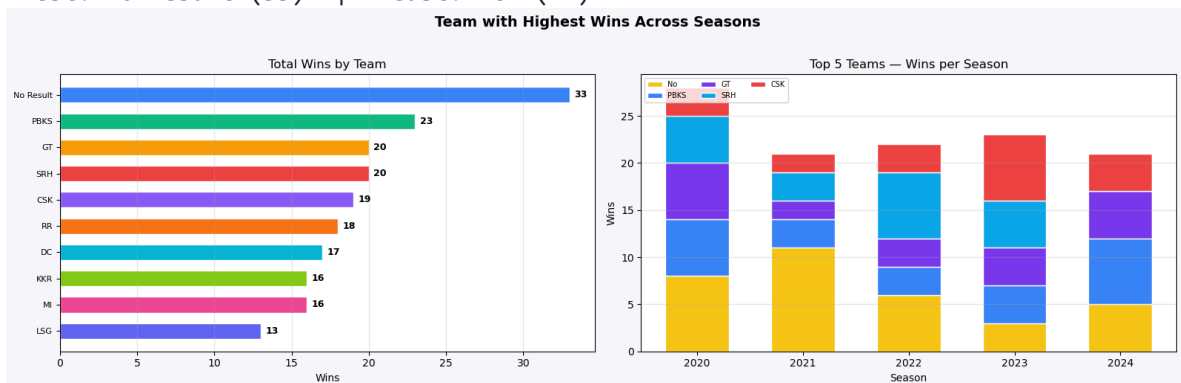
GT 20

SRH 20

CSK 19

dtype: int64

Best: No Result (33) | Least: RCB (11)



Step 6.2: Average Runs per Match

In [111]...

```
merged = deliveries.merge(matches[['match_id', 'season']], on='match_id')
per_match = merged.groupby(['season', 'match_id'])['batsman_runs'].sum().reset_index
per_match.columns = ['season', 'match_id', 'total_runs']
stats = per_match.groupby('season')['total_runs'].agg(['mean', 'median', 'std', 'min', 'max'])
print(f"Runs per match stats:\n{stats.round(2)}\nOverall mean: {per_match['total_runs'].mean()}")

fig, axes = plt.subplots(1, 2, figsize=(16, 5), facecolor='#F6F8FA')
fig.suptitle('Average Runs per Match', fontsize=14, fontweight='bold', y=1.02)
colors = ['#3B82F6', '#10B981', '#F59E0B', '#EF4444', '#8B5CF6']
seasons = sorted(per_match['season'].unique())

bp = axes[0].boxplot([per_match[per_match['season']==s]['total_runs'].values for s in seasons],
                    patch_artist=True, notch=False,
                    medianprops=dict(color='black', linewidth=2),
                    whiskerprops=dict(color='gray'), capprops=dict(color='gray'),
                    flierprops=dict(marker='o', color='gray', markersize=3))
[p.set(facecolor=c, alpha=0.7) for p, c in zip(bp['boxes'], colors)]
```

```

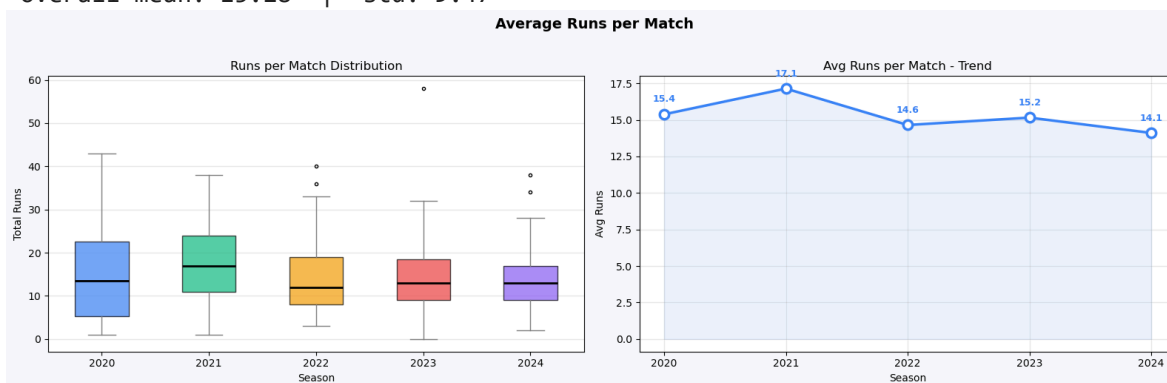
axes[0].set_xticks(range(1, len(seasons)+1)); axes[0].set_xticklabels(seasons)
axes[0].set(title='Runs per Match Distribution', xlabel='Season', ylabel='Total
axes[0].grid(axis='y', alpha=0.3); axes[0].set_facecolor('#FFFFFF')

avg = per_match.groupby('season')['total_runs'].mean().reset_index()
axes[1].plot(avg['season'], avg['total_runs'], color='#3B82F6', lw=2.5, marker='
            markersize=9, markerfacecolor='white', markeredgcolor='#3B82F6', m
axes[1].fill_between(avg['season'], avg['total_runs'], alpha=0.1, color='#3B82F6
[axes[1].annotate(f"{r['total_runs']:.1f}", (r['season'], r['total_runs'])),
            textcoords='offset points', xytext=(0,12), ha='center', fontsize=9, fontweight
        for _, r in avg.iterrows()]
axes[1].set_xticks(avg['season'])
axes[1].set(title='Avg Runs per Match - Trend', xlabel='Season', ylabel='Avg Run
axes[1].grid(alpha=0.3); axes[1].set_facecolor('#FFFFFF')
plt.tight_layout()
plt.show()

```

Runs per match stats:

	mean	median	std	min	max
season					
2020	15.38	13.5	10.43	1.0	43.0
2021	17.14	17.0	9.42	1.0	38.0
2022	14.65	12.0	9.40	3.0	40.0
2023	15.15	13.0	10.09	0.0	58.0
2024	14.11	13.0	7.69	2.0	38.0
Overall mean:	15.28		std: 9.47		



Step 6.3: No Result analysis

In [112...

```

matches['result'] = matches['winner'].apply(
    lambda w: 'No Result' if pd.isna(w) or str(w).strip()==' ' or str(w).strip().
total, no_res, comp = len(matches), (matches['result']=='No Result').sum(), (mat
print(f"Total: {total} | Completed: {comp} ({comp/total*100:.1f}%) | No Resu
print(f"No result per season:\n{matches[matches['result']=='No Result'].groupby(

fig, axes = plt.subplots(1, 2, figsize=(16, 5), facecolor='#F6F8FA')
fig.suptitle('Matches with No Result', fontsize=14, fontweight='bold', y=1.02)
summary = matches.groupby(['season', 'result']).size().unstack(fill_value=0)
seasons = summary.index.tolist(); x = np.arange(len(seasons)); w = 0.38

b1 = axes[0].bar(x-w/2, summary.get('Completed', pd.Series(0, index=seasons)).
b2 = axes[0].bar(x+w/2, summary.get('No Result', pd.Series(0, index=seasons))).va
[axes[0].text(b.get_x()+b.get_width()/2, b.get_height()+0.2, str(int(b.get_high
            ha='center', va='bottom', fontsize=9, fontweight='bold') for bars in [b1,b2] f
axes[0].set_xticks(x); axes[0].set_xticklabels(seasons)
axes[0].set(title='Completed vs No Result per Season', xlabel='Season', ylabel='
axes[0].legend(fontsize=9); axes[0].grid(axis='y', alpha=0.3); axes[0].set_faced

```

```

counts = matches['result'].value_counts()
comp_n, nores_n = counts.get('Completed',0), counts.get('No Result',0)
wedges, _ = axes[1].pie([comp_n, nores_n], colors=['#10B981', '#EF4444'],
                        startangle=90, wedgeprops=dict(width=0.5, edgecolor='wh
axes[1].text(0, 0, f"{nores_n/(comp_n+nores_n)*100:.1f}%\nNo Result", ha='center
            fontsize=13, fontweight='bold', color='#EF4444')
axes[1].legend(wedges, [f'Completed ({comp_n})', f'No Result ({nores_n})'],
              loc='lower center', fontsize=9, bbox_to_anchor=(0.5,-0.08), ncol=
axes[1].set(title='Overall No Result Proportion'); axes[1].set_facecolor('#FFFFF
plt.tight_layout(); plt.show()

```

Total: 206 | Completed: 173 (84.0%) | No Result: 33 (16.0%)

No result per season:

season

2020 8

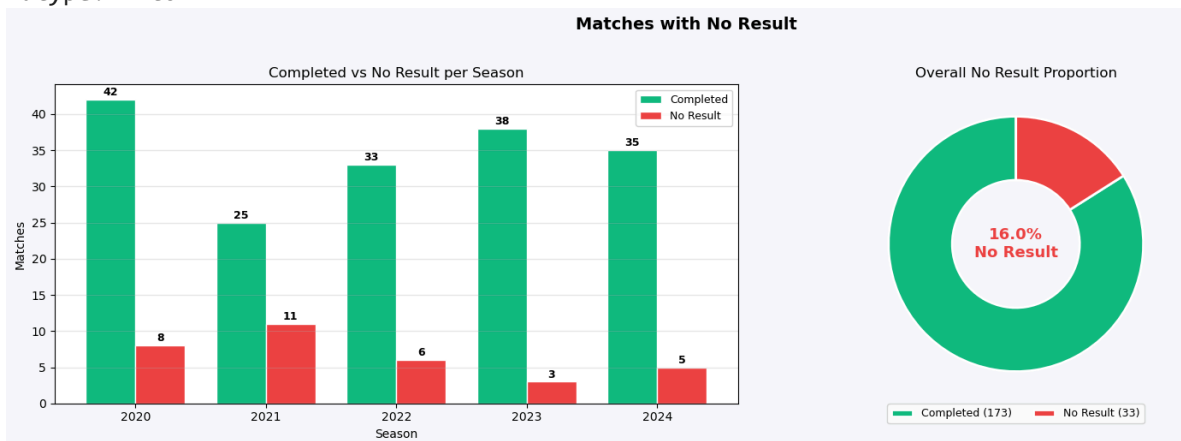
2021 11

2022 6

2023 3

2024 5

dtype: int64



Step 6.4: Ball Analytics

```

In [113... merged = deliveries.merge(matches[['match_id', 'season']], on='match_id')
over_avg = merged.groupby('over')['batsman_runs'].mean()
print(f"Avg runs per ball:\n{merged.groupby('ball')['batsman_runs'].mean().round
print(f"\nHighest over: {over_avg.idxmax()+1} | Lowest over: {over_avg.idxmin(

fig, axes = plt.subplots(1, 2, figsize=(16, 5), facecolor='#F6F8FA')
fig.suptitle('Player & Ball Analytics', fontsize=14, fontweight='bold', y=1.02)
colors = ['#3B82F6', '#10B981', '#F59E0B', '#EF4444', '#8B5CF6', '#F97316']

ball_avg = merged.groupby('ball')['batsman_runs'].mean().reset_index()
ball_avg = ball_avg[ball_avg['ball'].between(1,6)]
bars = axes[0].bar(ball_avg['ball'], ball_avg['batsman_runs'], color=colors, wid
[axes[0].text(b.get_x()+b.get_width()/2, b.get_height()+0.01, f"{v:.2f}",
             ha='center', va='bottom', fontsize=9, fontweight='bold') for b, v in zip(bars,
axes[0].set_xticks(ball_avg['ball']); axes[0].set_xticklabels([f'Ball {i}' for i
axes[0].set(title='Avg Runs per Ball Position (1-6)', xlabel='Ball in Over', yla
axes[0].grid(axis='y', alpha=0.3); axes[0].set_facecolor('#FFFFFF')

over_season = merged.groupby(['season', 'over'])['batsman_runs'].mean().unstack(f
im = axes[1].imshow(over_season.values, aspect='auto', cmap='YlOrRd')
axes[1].set_xticks(range(over_season.shape[1])); axes[1].set_xticklabels([str(i+

```



```
axes[1].set_yticks(range(len(over_season.index))); axes[1].set_yticklabels(over_season.index)
axes[1].set(title='Avg Runs Heatmap - Season x Over', xlabel='Over', ylabel='Season')
plt.colorbar(im, ax=axes[1], shrink=0.85, label='Avg Runs'); axes[1].set_facecolor('white')
plt.tight_layout(); plt.show()
```

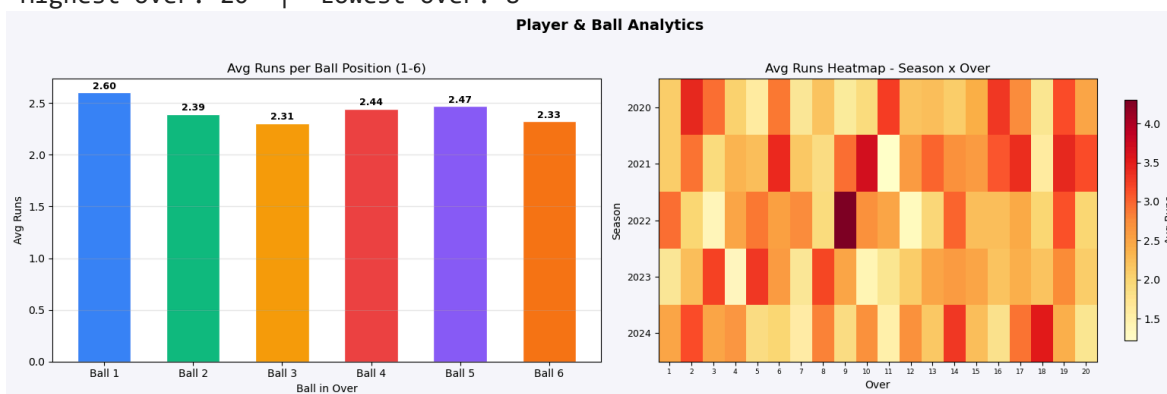
Avg runs per ball:

ball

```
1    2.604
2    2.393
3    2.306
4    2.444
5    2.469
6    2.325
```

Name: batsman_runs, dtype: float64

Highest over: 20 | Lowest over: 8



Step 6.5: Runs Distribution per Match

In [117...

```
merged = deliveries.merge(matches[['match_id', 'season']], on='match_id')
per_match = merged.groupby(['season', 'match_id'])['batsman_runs'].sum().reset_index()
per_match.columns = ['season', 'match_id', 'total_runs']
stats = per_match.groupby('season')['total_runs'].agg(['min', 'mean', 'max'])
stats.columns = ['Min', 'Mean', 'Max']
print(f"Min/Mean/Max per season:\n{stats.round(1)}\nHighest avg: {stats['Mean'].max()}")

fig, axes = plt.subplots(1, 2, figsize=(16, 5), facecolor='#F6F8FA')
fig.suptitle('Runs Distribution per Match', fontsize=14, fontweight='bold', y=1.05)
colors = ['#3B82F6', '#10B981', '#F59E0B', '#EF4444', '#8B5CF6']
seasons = sorted(per_match['season'].unique())

avg = per_match.groupby('season')['total_runs'].mean().reset_index()
bars = axes[0].bar(avg['season'], avg['total_runs'], color=colors[:len(avg)], width=0.8)
[axes[0].text(b.get_x()+b.get_width()/2, b.get_height()+2, f"{v:.0f}",
             ha='center', fontsize=10, fontweight='bold') for b, v in zip(bars, avg['total_runs'])]
axes[0].set_xticks(avg['season'])
axes[0].set(title='Avg Runs per Match by Season', xlabel='Season', ylabel='Avg Runs')
axes[0].grid(axis='y', alpha=0.3); axes[0].set_facecolor('white')

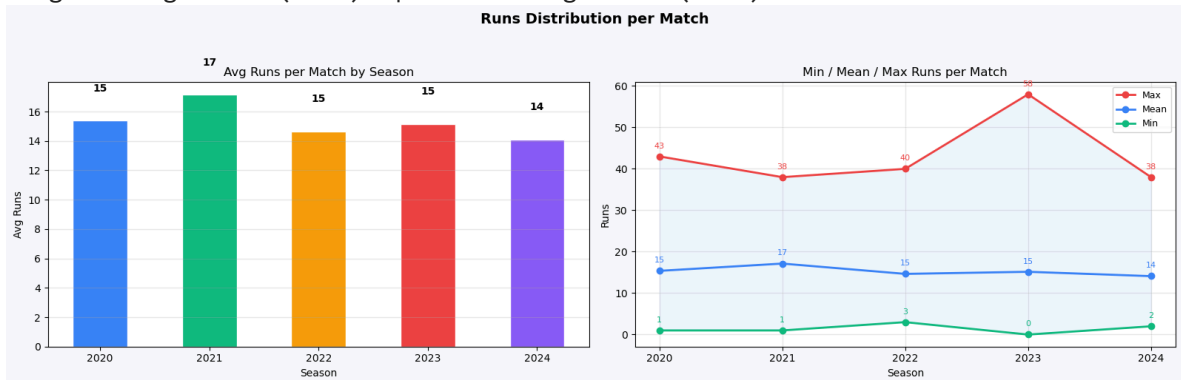
st = per_match.groupby('season')['total_runs'].agg(['min', 'mean', 'max']).reset_index()
for col, color in zip(['max', 'mean', 'min'], ['#EF4444', '#3B82F6', '#10B981']):
    axes[1].plot(st['season'], st[col], color=color, lw=2, marker='o', label=col)
    [axes[1].annotate(f"{r[col]:.0f}", (r['season'], r[col]), textcoords='offsetleft',
                    xytext=(0, 8), ha='center', fontsize=8, color=color) for _, r in st.iterrows()]
axes[1].fill_between(st['season'], st['min'], st['max'], alpha=0.08, color='#3B82F6')
axes[1].set_xticks(st['season'])
axes[1].set(title='Min / Mean / Max Runs per Match', xlabel='Season', ylabel='Runs')
```

```
axes[1].legend(fontsize=9); axes[1].grid(alpha=0.3); axes[1].set_facecolor('#FFF')
plt.tight_layout(); plt.show()
```

Min/Mean/Max per season:

	Min	Mean	Max
season			
2020	1.0	15.4	43.0
2021	1.0	17.1	38.0
2022	3.0	14.6	40.0
2023	0.0	15.2	58.0
2024	2.0	14.1	38.0

Highest avg: 2021 (17.1) | Lowest avg: 2024 (14.1)



Step 6.6: Wickets per Match

In [114...

```
merged = deliveries.merge(matches[['match_id', 'season']], on='match_id')
wpm = merged[merged['is_wicket']==1].groupby(['season', 'match_id']).size().reset
wpm.columns = ['season', 'match_id', 'wickets']
stats = wpm.groupby('season')['wickets'].agg(['mean', 'median', 'std', 'min', 'max'])
print(f"Wickets per match:\n{stats.round(2)}\nOverall avg: {wpm['wickets'].mean()}")

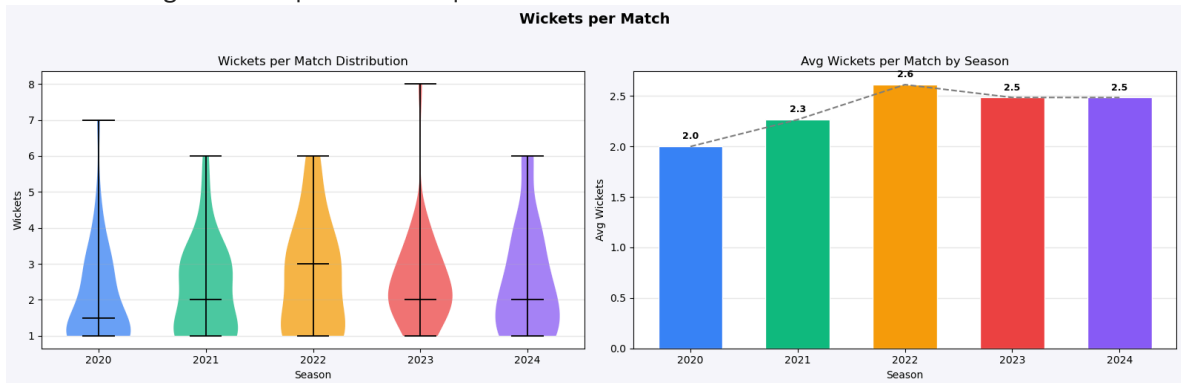
fig, axes = plt.subplots(1, 2, figsize=(16, 5), facecolor='#F6F8FA')
fig.suptitle('Wickets per Match', fontsize=14, fontweight='bold', y=1.02)
colors = ['#3B82F6', '#10B981', '#F59E0B', '#EF4444', '#8B5CF6']
seasons = sorted(wpm['season'].unique())

parts = axes[0].violinplot([wpm[wpm['season']==s]['wickets'].values for s in seasons],
                           positions=range(len(seasons)), showmedians=True, width=0.8)
[pc.set(facecolor=c, alpha=0.75) for pc, c in zip(parts['bodies'], colors)]
[parts[p].set(color='black', linewidth=1.3) for p in ('cmedians', 'cmins', 'cmaxes')]
axes[0].set_xticks(range(len(seasons))); axes[0].set_xticklabels(seasons)
axes[0].set(title='Wickets per Match Distribution', xlabel='Season', ylabel='Wickets')
axes[0].grid(axis='y', alpha=0.3); axes[0].set_facecolor('#FFFFFF')

avg = wpm.groupby('season')['wickets'].mean().reset_index()
bars = axes[1].bar(avg['season'], avg['wickets'], color=colors[:len(avg)], width=0.8)
[axes[1].text(b.get_x()+b.get_width()/2, b.get_height()+0.05, f"{v:.1f}",
              ha='center', va='bottom', fontsize=9, fontweight='bold') for b, v in zip(bars, avg['wickets'])]
axes[1].plot(avg['season'], avg['wickets'], color='gray', lw=1.5, ls='--', zorder=2)
axes[1].set_xticks(avg['season'])
axes[1].set(title='Avg Wickets per Match by Season', xlabel='Season', ylabel='Avg Wickets')
axes[1].grid(axis='y', alpha=0.3); axes[1].set_facecolor('#FFFFFF')
plt.tight_layout(); plt.show()
```

Wickets per match:

	mean	median	std	min	max
season					
2020	2.00	1.5	1.33	1	7
2021	2.27	2.0	1.34	1	6
2022	2.61	3.0	1.56	1	6
2023	2.49	2.0	1.31	1	8
2024	2.48	2.0	1.54	1	6
Overall avg: 2.35				Max: 8	Min: 1



Step 6.7: High-scoring Matches

```
In [115... merged = deliveries.merge(matches[['match_id','season']], on='match_id')
total = merged.groupby(['season','match_id'])['batsman_runs'].sum().reset_index()
total.columns = ['season','match_id','total_runs']
threshold = total['total_runs'].quantile(0.80)
high = total[total['total_runs'] >= threshold]
print(f"Threshold (80th pct): {threshold:.0f} | High-scoring matches: {len(high)}")
print(f"Per season:\n{high.groupby('season').size()}")

fig, axes = plt.subplots(1, 2, figsize=(16, 5), facecolor='#F6F8FA')
fig.suptitle('High-scoring Matches', fontsize=14, fontweight='bold', y=1.02)
colors = ['#3B82F6', '#10B981', '#F59E0B', '#EF4444', '#8B5CF6']
low_df, high_df = total[total['total_runs'] < threshold], total[total['total_runs'] >= threshold]

axes[0].scatter(low_df['season'] + np.random.uniform(-0.25,0.25,len(low_df)),
axes[0].scatter(high_df['season'] + np.random.uniform(-0.25,0.25,len(high_df)),
axes[0].axhline(threshold, color='#EF4444', ls='--', lw=1.2, alpha=0.6)
axes[0].set_xticks(sorted(total['season'].unique()))
axes[0].set(title='Match Scores - High-scoring Highlighted', xlabel='Season', y=1.02)
axes[0].legend(fontsize=8); axes[0].grid(alpha=0.3); axes[0].set_facecolor('#FFFFFF')

hs_df = pd.DataFrame({'count': high.groupby('season').size(), 'total': total.groupby('season').size()})
hs_df['pct'] = hs_df['count'] / hs_df['total'] * 100
bars = axes[1].bar(hs_df['season'], hs_df['count'], color=colors[:len(hs_df)], width=0.8)
[axes[1].text(b.get_x()+b.get_width()/2, b.get_height()+0.15, f"{int(r['count'])}%")
for b, (_, r) in zip(bars, hs_df.iterrows())]
axes[1].set_xticks(hs_df['season'])
axes[1].set(title='High-scoring Matches per Season', xlabel='Season', ylabel='Count', y=1.02)
axes[1].grid(axis='y', alpha=0.3); axes[1].set_facecolor('#FFFFFF')
plt.tight_layout(); plt.show()
```

Threshold (80th pct): 23 | High-scoring matches: 42 | Max: 58.0 | Median: 13

Per season:
season

2020 13

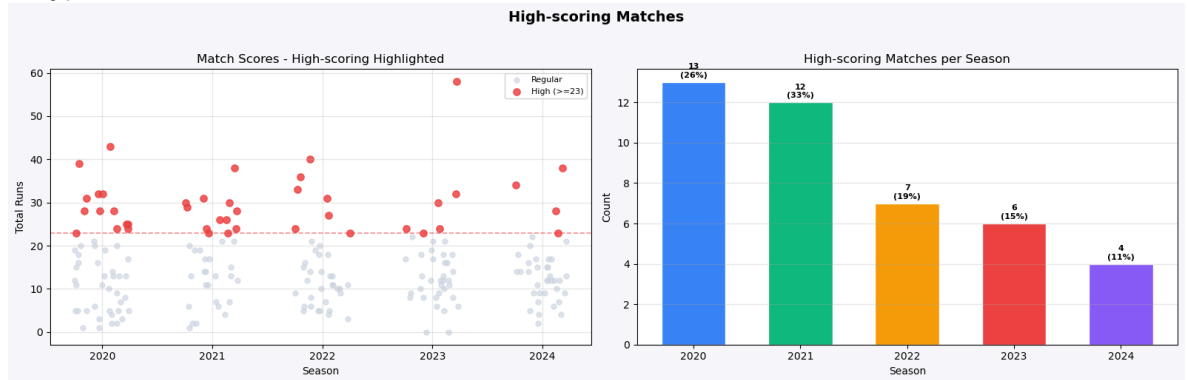
2021 12

2022 7

2023 6

2024 4

dtype: int64



In []:

In []:

In []:

In []:

In []:

In []:

In []: